

Session_11 activities worksheet

Online handouts: Printout of nonlinear.nb notebook, "Using GSL Interpolation Functions", listing of GslSpline and test files, ode_test.cpp listing, CL mystery guide

In this Session, we'll do a follow-up to Session_09, try out a GSL adaptive differential equation solver, briefly look at interpolation, take a first look at Python scripts for C++ programs, and do the "Command Line Mystery".

A) Follow-up to Session_09

Work on these tasks for the first third to half of the Session only, then move on to other Session_11 tasks.

1. The Mathematica notebook nonlinear.nb looks at the same type of analysis as diffeq_pendulum.cpp only it uses the Duffing equation. Look through it and follow along with the printout. This exercise will expose you to how to do lots of useful things with Mathematica, for future reference, but there's not much for you to do except run the notebook. *What questions do you have about Poincare sections or the power spectrum?*

→ I do not have any questions about Poincaré or power spectrum, since we went over it a lot in theoretical mechanics (Physics 5300).

If you look closely at the power spectrum, you'll see that there is a peak near zero frequency. Change the signal to be Fourier transformed to greatly reduce this peak. What did you do? (Hint: What would a component independent of frequency correspond to?)

→ A component independent of frequency would correspond to a system that doesn't cycle, so it has no frequency or frequency of zero. To greatly reduce this peak, we can transform the signal to center around 0 using the mean. This helps "kill" the power at the origin of the plot. We then apply Fourier to the transformed sample points.

2. If you're familiar with Mathematica and have time, you could convert nonlinear.nb to study the pendulum instead. *But for now, just run the "answer" notebook called pendulum.nb. Try different parameter choices; what do you see with p2 vs. p5? (For the Poincare plot with p5, try removing the PlotRange.)*

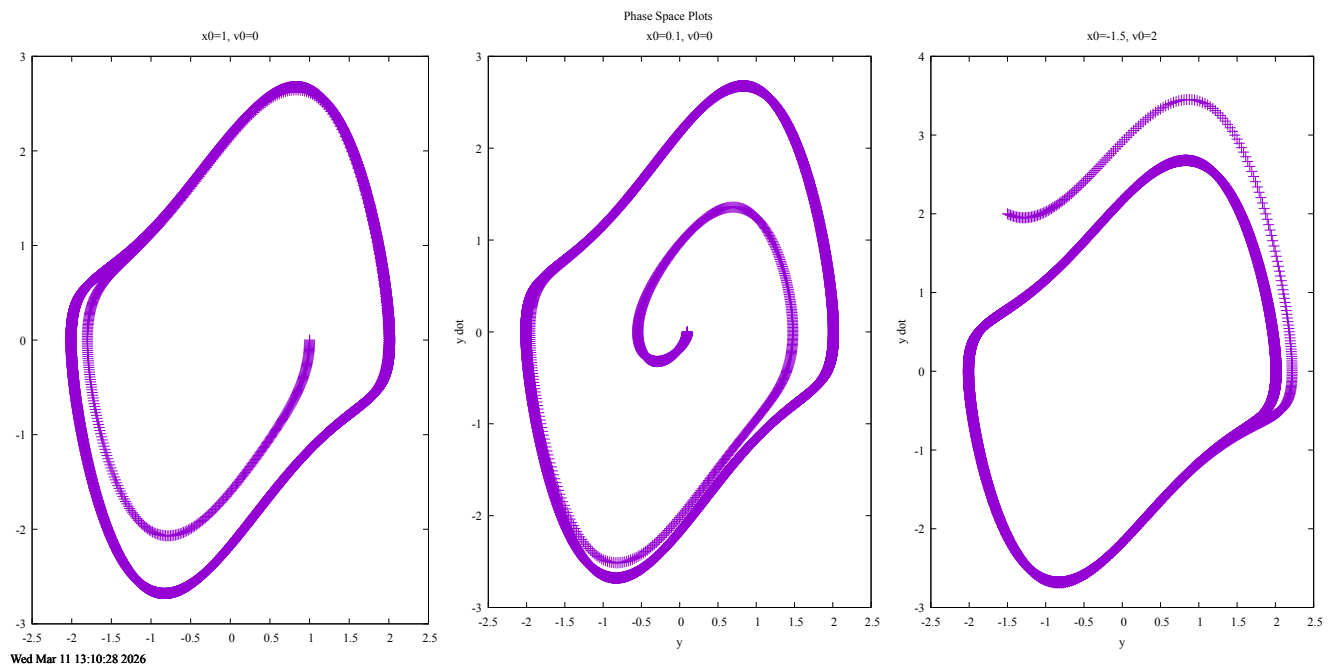
→ With p2, we see period 3 motion after the transient has died out, supported by the Poincaré section. With p5, we don't see good period motion. We see a more chaotic motion, which we can see with the Poincaré section that has multiple sporadic points instead of simply three points like p2.

B) GSL Differential Equation Solver

The program `ode_test.cpp` demonstrates the GSL adaptive differential equation solver by solving the Van der Pol oscillator, another nonlinear differential equation (see the Session_11 notes for the equation).

1. Take a look at the code and figure out where the values of μ and the initial conditions are set. Change μ to 2 and the initial conditions to $x_0=1.0$ and $v_0=0.0$ ($y[0]$ and $y[1]$). Note the different choices for "stepping algorithms", how the function is set up and that a Jacobian is defined, and how the equation is stepped along in time.
2. Use the makefile to compile and link the code. Run it.
3. Create three output files using the initial conditions $[x_0=1.0, v_0=0.0]$, $[x_0=0.1, v_0=0.0]$, and $[x_0=-1.5, v_0=2.0]$ (*just change values and recompile each time*). Notice how we've used a stringstream to uniquely name each file.
4. Use *gnuplot* to make phase-space plots of all three cases on a single plot, noting where they begin and end. Print it out and attach it. Describe what you observe? This is called an **isolated attractor**.

→ We see that the phase space plots for all three look very similar and seem to fall in the same attractor. Regardless of the initial conditions, the oscillator falls into the same general motion around the attractor, with minor differences based on the initial conditions. The plot is attached below.



5. Think about how you would restructure this code using classes. Next time we'll explore a possible implementation that is described in the Session_12 notes.

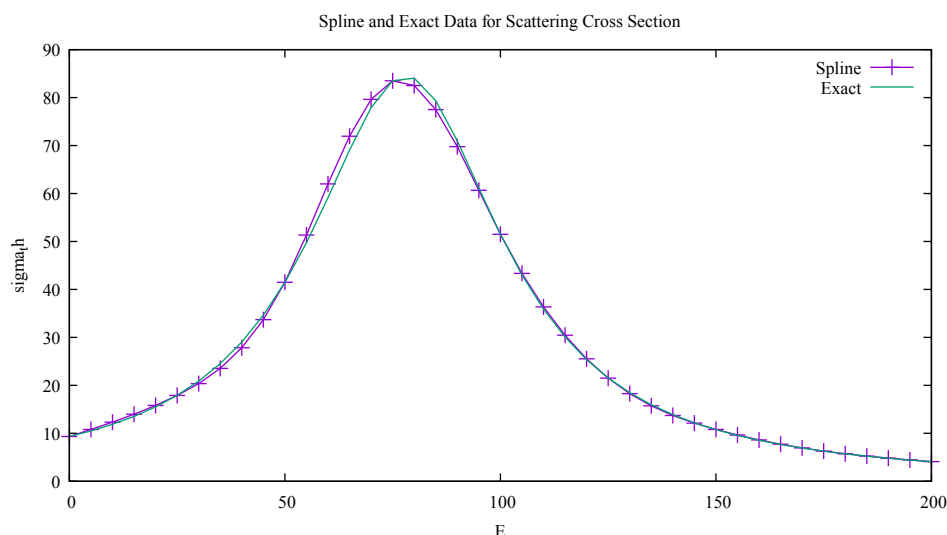
→ I have thought about it!

C) GSL Interpolation Routines

We'll use the example of a *theoretical* scattering cross section as a function of energy to try out the GSL interpolation routines. The (x,y) data, with $x \rightarrow E$ and $y \rightarrow \sigma_{th}$, is given in the bottom row of the table in the Session_11 notes (note we are NOT fitting σ_{exp}). You might think we should be doing this for the *experimental* cross section. Usually we will fit rather than interpolate such data because it is noisy and we also want to validate our interpolations against known functions.

1. Start with the `gsl_spline_test_class.cpp` code (and corresponding makefile). Take a look at the printout and try running the code. Note that we've used a Spline class as a "wrapper" for the GSL functions, just as we did earlier with the Hamiltonian class. Compare the implementation to the example on the "Using GSL Interpolation Functions" handout. *Questions?*
→ No questions, I think I understand the broad strokes of the implementation.
2. Instead of the sample function in the code, you will *change* the program to interpolate the data in the table from the notes. This will require deleting some of the code and adding new lines. Set `npts` and the (x,y) arrays equal to the appropriate values when you declare them. Declare them on separate lines. An array `x[4]` can be initialized with the values 1., 2., 3., and 4. with the declaration:
`double x[4] = {1., 2., 3., 4. };`
3. Use the code to generate a cubic spline interpolation for the cross section from 0 to 200 MeV in steps of 5 MeV. *Output this data and the exact results from equation (11.7) in the notes to a file for plotting with gnuplot and try it out. Plot the exact results "with lines" and the spline using "with linespoints" (or "w linesp"), so you can see both the individual points and the trends.*

→ Below is the plot.

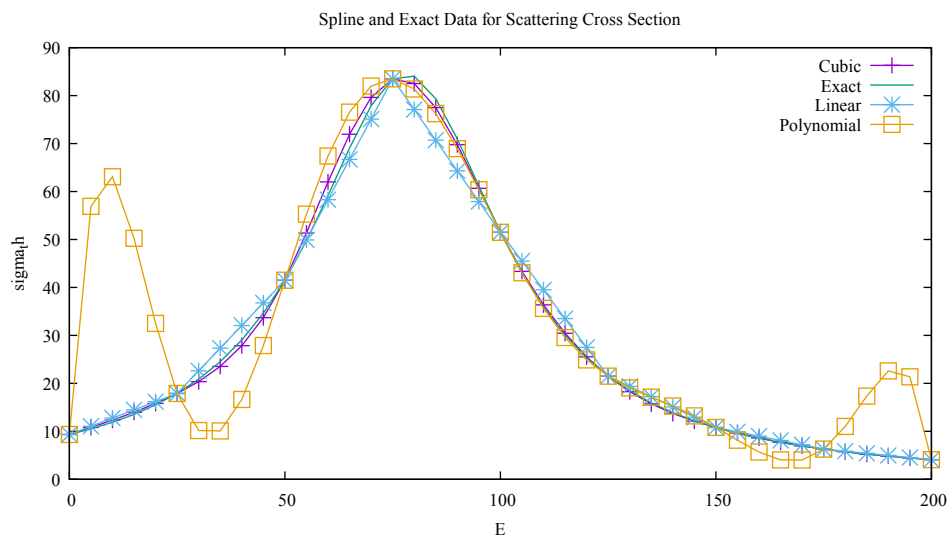


4. Now modify the Spline class to allow for a polynomial interpolation (see the GSL handout) and change the `gsl_spline_test_class.cpp` main program to generate linear and polynomial interpolations as well and add code to print the results to your output file. *Did you succeed?*

→ Yes, it worked.

5. Generate (and turn in) a graph with all three interpolations plotted along with the exact result. Comment here on the strengths and weaknesses of the different interpolation methods, both near the peak and globally.

→ Below is the plot. The cubic spline does the best job of approximating the exact function. The polynomial does the worst, especially when trying to fit the tails of function, which might be due to the fact that this function we're fitting is loosely related to $1/E^2$. The linear fit also does a good job of fitting, but since it has to be a straight line, it doesn't fit curved functions as well, causing it to not fit the smooth peak as well as the cubic spline. The discrete steps causes it to overshoot or undershoot values from the exact value.



D) Command Line Mystery

The "Command Line Mystery" is a whodunit designed to give you some practice with useful shell commands and how to string them together (with "pipes", which those of you who did it had a little bit of exposure to in Session_08 already). Follow the instructions on the `clmystery` handout. *Did you solve the mystery?*

→ Yes, I solved it. The answer is Jeremy Bowers.

E) Python Scripts for C++ Programs

This exercise is just a first exposure to what is possible with Python scripts. The listings for the scripts and revised versions of the area.cpp C++ programs are in the Session_11 notes.

1. Look at area_cmdline.cpp first and try it out (there is a makefile), first omitting an argument when executing it. Then look at and try run_area_cmdline1.py. *Change the list of numbers to generate the area for radii from 5 to 25 spaced by 5. Did you succeed?*
2. Modify both area_cmdline.cpp so that it takes *two* arguments, the radius and an integer called again. Change the code so the output line is repeated again times. Modify run_area_cmdline1.py so it works with this new version. *Did you succeed?*
3. Try out run_area_cmdline2.py, modifying value_list1 and value_list2 to help you understand how they work. *Questions?*
4. For now, just look through run_area_cmdline3.py and try running it. Note the use of findall and sorting, which may come in handy later.
5. Look at area_files.cpp and try it out (there is a makefile). There is also a Python script, run_area_files2.py, to try. (CHALLENGE) Modify the program and script so that the input file has an extra column for the integer again introduced in part 2.

→ Yes, it worked.

→ Yes, it worked.

→ No questions currently.

→ I made the changes for the integer addition.

F) Extra: Cubic Splining [if time permits]

Here we'll look at how to use cubic splines to define a function from arrays of x and y values. A question that always arises is: How many points do we need? Or, what may be more relevant, how accurate will our function (or its derivatives) be for a given spacing of x points?

1. We'll re-use the Spline class from above and the original gsl_spline_test_class.cpp function, which splined an array.
2. The goal is to modify the code so that it splines the ground-state hydrogen wave function: $u(r) = 2*r*\exp(-r)$
3. Your task is to determine how many (equally spaced) points to use to represent the wave function. Suppose you need the derivative of the wave function to be accurate to one part in 10^4 for $1 < r < 4$ (absolute, not relative error) *Devise (and carry out!) a plan that will tell you the spacing and the number of points needed to reach this goals. What did you do?*

→ My plan was the following: 1) Find the derivatives of $u(r)$ to identify the “worst case” of the

curvature within $1 < r < 4$. 2) Estimate the step size h using the error bound for derivatives. 3) Run the spline with some trial h and calculate the numerical derivative vs. the analytical derivative. 4) Determine N via $(r_{\text{max}} - r_{\text{min}})/h + 1$. There are more detailed comments for the application of this in the file where I carry this out “gsl_spline_test_class_copy.cpp”. The errors (absolute) in the spline get larger as you reach the boundaries ($r=1$ or $r=4$), so to ensure between $1 < r < 4$ we have less than $1e-4$ absolute error, we spline over a range like 0.9 to 4.1. Doing this, we get $h=0.006$ and $N=509$.

4. Now suppose you need integrals over the wave function to be accurate to 0.01%. *Devise (and carry out!) a plan that will tell you the spacing and the number of points needed to reach this goals.* To try out integrals, use one of the GSL integration routines on an integral involving the splined $u(r)$ that you know the answer to (hint: what is the total probability?). Note: The `qags_test.cpp` program from the Session_04 files can be quickly adapted for this exercise.

→ My plan was to do the following: 1) Take a reasonable r range of $0 < r < 15$ and a starting number of points $N=10$. 2) Continually increment the number of points (by 10) and populate the arrays for the r values and $u(r)$ values. 3) Give the values to the spline and set up a GSL workspace to integrate using QAGS. 4) Calculate the relative error and check if it's below the tolerance, if not increment N . Then from this, we can output the calculated N and h values. This is also in “gsl_spline_test_class_copy.cpp”. Doing this, we get $N=90$ and $h=0.17$.

G) Reflection

Write down which of the six learning goals on the syllabus this worksheet addressed for you and how:

→ This session helped me learn more of the tools of computational physics, like splines. I also gained a better understanding of physics problems, specifically with the power spectrum and Poincaré sections through code and computational solutions.